Instruções Entregável 1

🍦 Guia Oficial e Enunciado de Entrega: Checkpoint 1 (Gateway 1)

Disciplina: Algoritmos e Pensamento Computacional | **Público:** Calouros

Prazo de Validação: Dia 26 de outubro (em laboratório) (com extensão para a próxima Terça-feira) | **Valor:** 1,0 Ponto (da nota do Projeto P2)

Este documento reúne todas as instruções, regras de avaliação, orientações sobre o uso de Inteligência Artificial e um **exemplo completo de código em C** (baseado no tema fictício *"A Sorveteria Retrô dos Anos 80 — Ice Cream Palace"*) para guiar a sua equipe na entrega do **Checkpoint 1**.

🧠 1. A Regra do Arquiteto (Uso Consciente de IA e Autoria)

Sabemos que ferramentas de Inteligência Artificial (ChatGPT, Claude, Copilot, Gemini) fazem parte do dia a dia da tecnologia. Neste projeto, **vocês atuarão como Arquitetos de Software**. O uso de IA para tirar dúvidas ou auxiliar na estruturação é permitido, mas sob **3 regras inegociáveis**:

1. **Proibição de "Caixa-Preta" (Sintaxes Desconhecidas):** Na Prova P1, houve casos de códigos com comandos que nunca foram ensinados em aula (como typedef avançado, alocação dinâmica complexa ou bibliotecas externas) que o aluno não fazia ideia do que significavam. **Priorizem usar as estruturas ensinadas em aula** (struct padrão, vetores globais, if/else, switch-case, for, while, do-while e funções).
2. **Responsabilidade Total sobre cada Linha:** Se uma linha de código está no projeto do seu grupo, você tem a obrigação de saber explicar **o que ela faz** e **por que ela foi escrita daquela forma**.
3. **Arguição Técnica:** Nos Checkpoints presenciais o professor poderá apontar para qualquer if, laço for, parâmetro de função ou campo de struct do seu módulo e perguntar: *"Por que você usou isso aqui?"*. Se o integrante não souber explicar o código que traz o nome dele, a nota individual daquele critério será impactada.

💡 **Nota sobre o futuro do projeto:** Neste Checkpoint 1, nossos bancos de dados serão vetores de structs de tamanho fixo na memória RAM. Nas próximas aulas, aprenderemos **Leitura e Escrita de Arquivos (FILE *)** para salvar e carregar esses mesmos vetores em arquivos .txt ou .csv!

🏛️ 2. Como Dividir o Projeto na Equipe (Perspectivas de Domínio)

Em vez de uma pessoa fazer apenas "o cadastro" e outra fazer apenas "a busca", **cada integrante do grupo (3 a 4 alunos) será dono de uma Perspectiva (Entidade/Módulo) completa do sistema** dentro do mesmo arquivo main.c (modo console no Dev-C++ ou VS Code).

Dentro da sua perspectiva, o aluno deve modelar:

- A sua própria **struct** no escopo global (com pelo menos 4 campos heterogêneos).
- O seu **Submenu Interativo** (função void chamada pelo menu principal) contendo a estrutura para as **4 operações fundamentais (CRUD)**:

1. **Criar / Cadastrar** novo registro.
2. **Listar / Buscar** registros existentes.
3. **Editar / Atualizar** dados de um registro.
4. **Remover / Excluir** registro (ou dar baixa).

(No 4º módulo — ou dividido entre o grupo caso sejam 3 alunos — entram também as rotinas de Movimentação/Vendas e os Relatórios Analíticos, como "qual o cliente com mais compras" ou "qual o item mais vendido" por exemplo).

📋 3. Rubrica de Avaliação do Checkpoint 1 (Valor Total: 1,0 Ponto)

Para conquistar o **1,0 ponto** integral na terça-feira, o projeto da equipe será avaliado nos seguintes 4 critérios objetivos (as pontuações dos subtópicos dizem respeito a como será pontuado o tópico de acordo com a qualidade da entrega, não são somatórias):

- **Critério 1.1: Repositório no GitHub e Organização (Valor: 0,25 pt)**
- **0,25 pt:** Repositório criado no GitHub, compartilhado com a equipe e o professor (**rduarte@up.edu.br (mailto:rduarte@up.edu.br)**)contendo os primeiros *commits* da estrutura base feitos ao longo da semana.
- **0,0 pt:** Sem repositório no GitHub (código levado apenas em arquivo solto ou pen drive).
- **Critério 1.2: Modelagem das structs por Perspectiva (Valor: 0,25 pt)**
- **0,25 pt:** structs de cada integrante declaradas corretamente no **Escopo Global**, contendo pelo menos **4 campos de tipos variados** (int, float, char[]) coerentes com o tema retrô da equipe, acompanhadas de seus vetores globais de armazenamento ([50]).
- **0,10 pt:** structs criadas com menos de 4 campos, tipos inadequados ou declaradas presas dentro do main().
- **0,0 pt:** Ausência de struct (uso apenas de variáveis soltas).
- **Critério 1.3: Menu Principal e Submenus de Perspectiva (Valor: 0,25 pt)**
- **0,25 pt:** Menu principal no main() operando em loop contínuo (do-while + switch-case) que direciona para os **Submenus de cada integrante**, onde já constam as opções de Cadastrar, Buscar/Listar, Editar e Remover.
- **0,10 pt:** Menu roda apenas uma vez e fecha o programa, ou não apresenta os submenus das perspectivas de cada aluno.
- **0,0 pt:** Sem menu interativo funcional.
- **Critério 1.4: Modularização em Funções e Identificação de Autoria (Valor: 0,25 pt)**
- **0,25 pt:** Código organizado em funções (void) separadas fora do main(), todas identificadas no cabeçalho com o // Autor: Nome do Aluno e compilando com **zero erros** no Dev-C++ / GCC.

- **0,10 pt:** Todo o código misturado dentro do main() sem divisão em funções ou sem identificação de qual integrante fez cada parte.
- **0,0 pt:** Código não compila.
-

💻 4. Exemplo Prático Completo REDUZIDO: "A Sorveteria dos Anos 80 (Ice Cream Palace)"

Abaixo está o modelo exato do que você e seu grupo devem construir para o tema para a próxima entrega (teremos parte do horário de aula para avançar nessa atividade). Observe como **não é necessário ter todas as contas e lógicas prontas no Checkpoint 1**, mas toda a **arquitetura de structs, menus, submenus e divisão por integrante** já deve estar rodando perfeitamente no terminal:

```c
#include <stdio.h>
#include <string.h>

// ================================================================================
// PARTE 1: MODELAGEM DE DADOS (ESCOPO GLOBAL - 1 STRUCT POR INTEGRANTE)
// ================================================================================

// 1. Perspectiva do Integrante A: Clientes do Clube do Sorvete
struct Cliente {
    int idCliente;
    char nome[50];
    char telefone[20];
    int fichasAcumuladas;
    int ativo; // 1 = Ativo, 0 = Removido
};

struct Cliente listaClientes[50];
int totalClientes = 0;

// 2. Perspectiva do Integrante B: Catálogo de Sabores (Massas)
struct SaborSorvete {
    int idSabor;
    char nomeSabor[40];
    float precoBola;
    float litrosEstoque;
    int disponivel; // 1 = Disponível, 0 = Esgotado
};
struct SaborSorvete catalogoSabores[50];
int totalSabores = 0;

// 3. Perspectiva do Integrante C: Estoque de Coberturas e Insumos
struct Insumo {
    int idInsumo;
    char descricao[40];
    char fornecedor[40];
    float custoCompra;
    int quantidadePacotes;
};
struct Insumo estoqueInsumos[50];
int totalInsumos = 0;

// 4. Perspectiva do Integrante D: Vendas no Balcão e Relatórios
struct Venda {
    int idVenda;
    int idClienteComprador;
    int idSaborEscolhido;
    int quantidadeBolas;
    float valorTotalPedido;
```

```
};
struct Venda historicoVendas[50];
int totalVendas = 0;


// ============================================================================
// PARTE 2: FUNÇÕES DE INTERFACE E MÓDULOS DE CADA INTEGRANTE
// ============================================================================

// Função Visual Comum da Equipe
void exibirCabecalho() {
    printf("\n========================================================\n");
    printf("   ICE CREAM PALACE - SISTEMA DE GESTAO RETRO (1985)  \n");
    printf("========================================================\n");
}

// ----------------------------------------------------------------------------
// MÓDULO 1: PERSPECTIVA DE CLIENTES
// Autor: Integrante A (Nome Completo e RGM)
// ----------------------------------------------------------------------------
void moduloClientes() {
    int opcaoCliente;
    do {
        printf("\n--- [MODULO 1: GESTAO DE CLIENTES (Integrante A)] ---\n");
        printf("1. Cadastrar Novo Cliente\n");
        printf("2. Listar / Buscar Clientes\n");
        printf("3. Editar Dados ou Fichas de Cliente\n");
        printf("4. Remover Cliente\n");
        printf("0. Voltar ao Menu Principal\n");
        printf("Escolha uma acao: ");
        scanf("%d", &opcaoCliente);

        switch (opcaoCliente) {
            case 1:
                printf("\n[Em construcao] Rotina de Cadastro de Cliente...\n");
                printf("Campos previstos: ID, Nome, Telefone e Fichas Iniciais.\n");
                break;
            case 2:
                printf("\n[Em construcao] Rotina de Busca e Listagem de
Clientes...\n");
                break;
            case 3:
                printf("\n[Em construcao] Rotina de Edicao de Cliente por ID...\n");
                break;
            case 4:
                printf("\n[Em construcao] Rotina de Remocao de Cliente...\n");
                break;
            case 0:
```

```
                printf("\nRetornando ao menu principal...\n");
                break;
            default:
                printf("\n[ERRO] Opcao invalida no Modulo de Clientes!\n");
        }
    } while (opcaoCliente != 0);
}

// --------------------------------------------------------------------------
// MÓDULO 2: PERSPECTIVA DE PRODUTOS / SABORES DE SORVETE
// Autor: Integrante B (Nome Completo e RGM)
// --------------------------------------------------------------------------
void moduloSabores() {
    int opcaoSabor;
    do {
        printf("\n--- [MODULO 2: CATALOGO DE SABORES (Integrante B)] ---\n");
        printf("1. Cadastrar Novo Sabor de Sorvete\n");
        printf("2. Listar Cardapio / Buscar Sabor\n");
        printf("3. Alterar Preco ou Estoque de Sabor\n");
        printf("4. Excluir Sabor do Catalogo\n");
        printf("0. Voltar ao Menu Principal\n");
        printf("Escolha uma acao: ");
        scanf("%d", &opcaoSabor);

        switch (opcaoSabor) {
            case 1:
                printf("\n[Em construcao] Cadastrando Sabor (ID, Nome, Preco da Bola,
Litros)...\n");
                break;
            case 2:
                printf("\n[Em construcao] Listando todos os sabores
disponiveis...\n");
                break;
            case 3:
                printf("\n[Em construcao] Atualizando preco ou litros em
estoque...\n");
                break;
            case 4:
                printf("\n[Em construcao] Removendo sabor fora de linha...\n");
                break;
            case 0:
                printf("\nRetornando ao menu principal...\n");
                break;
            default:
                printf("\n[ERRO] Opcao invalida no Modulo de Sabores!\n");
        }
    } while (opcaoSabor != 0);
}
```

```
// ------------------------------------------------------------------------
// MÓDULO 3: PERSPECTIVA DE COMPRA DE MATÉRIA-PRIMA / INSUMOS
// Autor: Integrante C (Nome Completo e RGM)
// ------------------------------------------------------------------------
void moduloInsumos() {
    int opcaoInsumo;
    do {
        printf("\n--- [MODULO 3: ESTOQUE DE INSUMOS E COBERTURAS (Integrante C)] ---
\n");
        printf("1. Registrar Compra de Matéria-Prima / Cobertura\n");
        printf("2. Consultar Estoque de Insumos\n");
        printf("3. Atualizar Quantidade de Pacotes / Custo\n");
        printf("4. Dar Baixa / Remover Insumo Vencido\n");
        printf("0. Voltar ao Menu Principal\n");
        printf("Escolha uma acao: ");
        scanf("%d", &opcaoInsumo);

        switch (opcaoInsumo) {
            case 1:
                printf("\n[Em construcao] Entrada de nota de insumos (Casquinhas,
Caldas)...\n");
                break;
            case 2:
                printf("\n[Em construcao] Listando estoque por fornecedor...\n");
                break;
            case 3:
                printf("\n[Em construcao] Editando pacotes em estoque...\n");
                break;
            case 4:
                printf("\n[Em construcao] Baixa de insumo no almoxarifado...\n");
                break;
            case 0:
                printf("\nRetornando ao menu principal...\n");
                break;
            default:
                printf("\n[ERRO] Opcao invalida no Modulo de Insumos!\n");
        }
    } while (opcaoInsumo != 0);
}

// ------------------------------------------------------------------------
// MÓDULO 4: PERSPECTIVA DE VENDAS E RELATÓRIOS ANALÍTICOS (CRUZAMENTO DE DADOS)
// Autor: Integrante D (Nome Completo e RGM)
// ------------------------------------------------------------------------
void moduloVendasRelatorios() {
    int opcaoVenda;
    do {
```

```
        printf("\n--- [MODULO 4: VENDAS E RELATORIOS ANALITICOS (Integrante D)] ---
\n");
        printf("1. Registrar Nova Venda no Balcao (Cruza Cliente + Sabor)\n");
        printf("2. Listar Historico de Vendas do Dia\n");
        printf("3. Cancelar / Estornar Venda\n");
        printf("4. [RELATORIO] Sabor com Maior Demanda (Mais Vendido)\n");
        printf("5. [RELATORIO] Cliente VIP (Maior Numero de Compras)\n");
        printf("6. [RELATORIO] Compras Realizadas por um Cliente Especifico\n");
        printf("0. Voltar ao Menu Principal\n");
        printf("Escolha uma acao: ");
        scanf("%d", &opcaoVenda);

        switch (opcaoVenda) {
            case 1:
                printf("\n[Em construcao] Lancamento de pedido vinculando ID Cliente e
ID Sabor...\n");
                break;
            case 2:
                printf("\n[Em construcao] Exibindo todas as vendas
registradas...\n");
                break;
            case 3:
                printf("\n[Em construcao] Estorno de venda por ID...\n");
                break;
            case 4:
                printf("\n[Em construcao] Calculando ranking do sabor mais
pedido...\n");
                break;
            case 5:
                printf("\n[Em construcao] Identificando cliente com maior volume de
compras...\n");
                break;
            case 6:
                printf("\n[Em construcao] Filtrando sorvetes comprados por um ID de
cliente...\n");
                break;
            case 0:
                printf("\nRetornando ao menu principal...\n");
                break;
            default:
                printf("\n[ERRO] Opcao invalida no Modulo de Vendas!\n");
        }
    } while (opcaoVenda != 0);
}


// ========================================================================
// PARTE 3: FUNÇÃO PRINCIPAL (MAIN) - DESPACHANTE DE MENUS
```

```
// ============================================================================
int main() {
    int opcaoPrincipal;

    do {
        exibirCabecalho();
        printf("SELECIONE A PERSPECTIVA DO SISTEMA:\n");
        printf("1. Modulo de Clientes (Integrante A)\n");
        printf("2. Modulo de Sabores / Cardapio (Integrante B)\n");
        printf("3. Modulo de Insumos / Materia-Prima (Integrante C)\n");
        printf("4. Modulo de Vendas e Relatorios (Integrante D)\n");
        printf("0. Encerrar o Sistema\n");
        printf("-------------------------------------------------------\n");
        printf("Digite a opcao desejada: ");
        scanf("%d", &opcaoPrincipal);

        switch (opcaoPrincipal) {
            case 1:
                moduloClientes();
                break;
            case 2:
                moduloSabores();
                break;
            case 3:
                moduloInsumos();
                break;
            case 4:
                moduloVendasRelatorios();
                break;
            case 0:
                printf("\nEncerrando o sistema Ice Cream Palace... Ate logo!\n");
                break;
            default:
                printf("\n[ALERTA] Opcao invalida! Digite um numero de 0 a 4.\n");
        }
    } while (opcaoPrincipal != 0);

    return 0;
}
```

✅ Checklist Rápido para a terça-feira (Validação em Sala)

Antes de chamar o professor na bancada na terça-feira, confira se o grupo cumpre todos os itens abaixo:

- [ ] O repositório do GitHub está criado e o código está salvo lá?

- [ ] Cada integrante criou a sua struct no escopo global com pelo menos 4 campos?
- [ ] O menu principal abre os 4 submenus (um de cada integrante) e permite voltar ao menu inicial digitando 0 sem fechar o programa acidentalmente?
- [ ] Cada submenu lista claramente as 4 ações básicas (Cadastrar, Listar/Buscar, Editar, Remover) e o módulo de relatórios lista os cruzamentos analíticos?
- [ ] Todos os integrantes sabem explicar exatamente o que faz cada linha do código entregue?